

Độ phức tạp thuật toán, phương pháp so sánh độ phức tạp, xác định hàm tính độ phức tạp

1 Khái niệm chung

Độ phức tạp thuật toán (Algorithm Complexity) là một đại lượng dùng để đo hiệu quả của thuật toán về thời gian thực thi (Time Complexity) và bộ nhớ sử dụng (Space Complexity), phụ thuộc vào kích thước đầu vào của bài toán.

Mục đích của việc xác định độ phức tạp là để so sánh và lựa chọn thuật toán tối ưu khi giải quyết cùng một vấn đề.

2 Các phương pháp so sánh độ phức tạp

2.1 Dựa vào bậc tăng trưởng của hai hàm số

Nếu hai thuật toán có hàm biểu diễn độ phức tạp lần lượt là $f(n)$ và $g(n)$, ta so sánh tốc độ tăng trưởng của chúng khi $n \rightarrow \infty$.

- $f(n) = n^2, g(n) = n^3 \Rightarrow g(n)$ tăng nhanh hơn $\Rightarrow$ thuật toán có độ phức tạp $O(n^3)$ kém hiệu quả hơn.

2.2 Dựa vào giới hạn (lim) để đánh giá tiệm cận

Ta dùng giới hạn để so sánh trực tiếp:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$$

- Nếu giới hạn $= 0 \Rightarrow f(n) = o(g(n)) \Rightarrow f(n)$ nhỏ hơn $g(n)$.
- Nếu giới hạn là hằng số khác 0 $\Rightarrow f(n) = \Theta(g(n)) \Rightarrow$ tương đương nhau.
- Nếu giới hạn $\rightarrow \infty \Rightarrow f(n) = \omega(g(n)) \Rightarrow f(n)$ lớn hơn $g(n)$.

2.3 Dựa vào các lớp tương đương (class of growth)

Các thuật toán được chia thành nhóm có tốc độ tăng tương đương:

Lớp	Biểu thức	Đặc điểm
Hằng	$O(1)$	Thời gian không đổi
Logarithmic	$O(\log n)$	Tăng chậm, rất hiệu quả
Tuyến tính	$O(n)$	Tăng theo kích thước đầu vào
Tuyến tính-log	$O(n \log n)$	Hiệu quả cho sắp xếp
Bậc hai	$O(n^2)$	Thường gặp ở thuật toán lồng nhau
Mũ	$O(2^n), O(n!)$	Rất kém hiệu quả

3 Cách xác định hàm tính độ phức tạp

3.1 Bước 1: Xác định tham số quyết định

Tham số này thường là **kích thước đầu vào** n , ví dụ:

- Số phần tử trong mảng
- Số đỉnh hoặc cạnh trong đồ thị
- Kích thước ma trận

3.2 Bước 2: Đếm số phép toán cơ bản

Tính số lần thực hiện các phép cơ sở (gán, cộng, so sánh, gọi hàm, ...). Mục tiêu là xác định hàm $f(n)$ biểu diễn số phép toán phụ thuộc vào n .

3.3 Bước 3: Áp dụng các quy tắc tính độ phức tạp

1. Quy tắc bỏ hằng số

Bỏ qua các hệ số hoặc giá trị cố định:

$$T(n) = 3n + 10 \Rightarrow O(n)$$

2. Quy tắc lấy max

Trong các phần cộng có bậc khác nhau, giữ lại bậc lớn nhất:

$$T(n) = n^2 + n + 100 \Rightarrow O(n^2)$$

3. Quy tắc cộng

Nếu có hai phần độc lập, tổng độ phức tạp bằng phần có độ phức tạp lớn hơn:

$$T(n) = O(f(n)) + O(g(n)) \Rightarrow O(\max(f(n), g(n)))$$

4. Quy tắc nhân

Khi một phần phụ thuộc vào phần khác (vòng lặp lồng nhau):

$$T(n) = O(f(n) \times g(n))$$

Ví dụ: Hai vòng lặp lồng nhau mỗi vòng chạy n lần $\Rightarrow O(n^2)$.

4 Dựa vào ký hiệu tiệm cận

Big O — Giới hạn trên

Cho biết tốc độ tăng trưởng **tối đa** của hàm, dùng để đánh giá **độ phức tạp tệ nhất** (Worst Case).

Big Ω (Omega) — Giới hạn dưới

Cho biết tốc độ tăng trưởng **tối thiểu**, mô tả **trường hợp tốt nhất** (Best Case).

Big Θ (Theta) — Giới hạn chặt

Khi thuật toán có cùng tốc độ tăng trưởng ở cả trên và dưới, tức là ước lượng **chính xác**:

$$f(n) = \Theta(g(n)) \Leftrightarrow O(g(n)) \text{ và } \Omega(g(n))$$

5 Vì sao lấy từ n_0 chứ không phải từ 0

- Khi n nhỏ, dữ liệu chưa đủ lớn để phản ánh bản chất tăng trưởng thật sự.
- Với $n < n_0$, sai số giữa các thuật toán có thể do chi phí khởi tạo hoặc yếu tố phụ.
- Vì vậy, việc so sánh chỉ có ý nghĩa khi $n \geq n_0$, tức là dữ liệu đủ lớn để tính chất tiệm cận được thể hiện rõ ràng.

Tóm tắt

Bước	Nội dung chính
1	Xác định tham số quyết định (kích thước đầu vào)
2	Tính số phép toán cơ bản
3	Áp dụng 4 quy tắc tính độ phức tạp
4	Biểu diễn bằng ký hiệu tiệm cận (O , Ω , Θ)
5	So sánh bằng tốc độ tăng trưởng hoặc giới hạn lim
6	Chỉ xét khi $n \geq n_0$ để đảm bảo tính tiệm cận đúng

Kết luận

Việc xác định và so sánh độ phức tạp giúp ta hiểu rõ hiệu suất thuật toán, tối ưu hóa chương trình và lựa chọn giải pháp phù hợp với quy mô dữ liệu thực tế.